1. PASAJE DE PARÁMETROS

El pasaje de parámetros permite que un programa llame a un procedimiento y le entregue datos para que los utilice y, eventualmente, devuelva resultados.

Los parámetros pueden pasarse principalmente mediante:

- **Registros:** el programa que llama coloca los parámetros en registros acordados y el procedimiento llamado los utiliza. Es rápido porque evita accesos a memoria, pero está limitado por la cantidad de registros.
- **Memoria:** se reserva una zona conocida por ambos procedimientos para almacenar los parámetros y resultados. Es más flexible en cantidad, pero implica accesos a memoria y hay que manejar correctamente las zonas para llamadas anidadas.
- **Pila:** los parámetros se almacenan en la pila junto con información de la llamada, formando parte del **marco de pila**. Es flexible y se adapta bien a llamadas anidadas porque la pila sigue una organización LIFO.

También puede hablarse de **pasaje por valor**, cuando se entrega el valor del dato, y **por referencia**, cuando se entrega una referencia o dirección que permite trabajar sobre el dato original.

2. MECANISMO GENERAL DE INTERRUPCIONES

Una **interrupción** permite que un dispositivo o evento solicite la atención de la CPU para que el procesador interrumpa temporalmente el programa que estaba ejecutando y ejecute una **rutina de servicio o gestor de interrupción**.

El mecanismo general es:

1. Un dispositivo genera una **petición de interrupción**.
2. El procesador termina la instrucción que corresponda y comprueba si existe una interrupción pendiente y habilitada.
3. Se **suspende el programa actual y se guarda su contexto**, principalmente la dirección de retorno y el estado del procesador.
4. Se carga en el **PC** la dirección de la rutina de servicio de interrupción.
5. Se ejecuta dicha rutina, que determina la causa de la interrupción y realiza las acciones necesarias.
6. Al terminar, se recupera el contexto guardado.
7. Se restaura el estado del procesador y continúa la ejecución del programa interrumpido.

La ventaja es que el procesador no necesita estar consultando constantemente a los dispositivos. Puede ejecutar otras instrucciones mientras espera que un dispositivo necesite atención. Escuela ESAM

Una diferencia importante con una llamada a subrutina es que **la interrupción no fue necesariamente provocada por el programa en ejecución y puede ocurrir en cualquier momento**, por eso es necesario guardar el contexto del programa interrumpido.

3. INTERRUPCIONES MÚLTIPLES

Interrupciones múltiples

Las **interrupciones múltiples** aparecen cuando pueden existir varias solicitudes de interrupción al mismo tiempo o cuando llega una nueva interrupción mientras se está atendiendo otra.

Las interrupciones pueden clasificarse en **enmascarables y no enmascarables**. Las enmascarables pueden ser deshabilitadas mediante el mecanismo de habilitación/máscara del procesador, mientras que las no enmascarables deben ser reconocidas.

Para tratar varias interrupciones, Stallings presenta dos alternativas. La primera es **deshabilitar nuevas interrupciones mientras se atiende una actual**. Las nuevas solicitudes generalmente quedan pendientes y se atienden cuando las interrupciones vuelven a habilitarse. Es un método simple y permite un tratamiento secuencial, pero no tiene en cuenta la prioridad ni las interrupciones que requieren atención rápida.

La segunda alternativa es utilizar **prioridades y anidamiento**. Una interrupción de mayor prioridad puede interrumpir al gestor que está atendiendo una interrupción de menor prioridad. En ese caso se guarda el contexto de la rutina interrumpida para poder retomarla después. Cuando termina la interrupción de mayor prioridad, se restaura el estado anterior y continúa la rutina que había quedado suspendida.

El objetivo de utilizar prioridades es que **los eventos más urgentes sean atendidos antes que los de menor prioridad**, evitando, por ejemplo, que un dispositivo con datos que llegan rápidamente pierda información por esperar demasiado.

4. Reconocimiento de interrupciones vectorizadas mediante PIC

En una interrupción vectorizada, el dispositivo que solicita la interrupción puede ser identificado mediante un **vector**, que permite localizar la rutina de servicio correspondiente.

En el caso del **PIC 82C59A**, el mecanismo es el siguiente:

1. Un dispositivo externo genera una solicitud de interrupción y la envía a una de las entradas **IR0–IR7** del PIC.
2. El PIC recibe las solicitudes, determina cuál tiene mayor prioridad y, si corresponde atenderla, activa la señal **INTR** hacia el procesador.
3. La CPU termina la instrucción que estaba ejecutando y reconoce la interrupción mediante **INTA (Interrupt Acknowledge)**.
4. Al recibir el reconocimiento, el PIC coloca en el **bus de datos el vector de interrupción correspondiente**.
5. El procesador utiliza ese vector para localizar la rutina de servicio y guarda el contexto del programa interrumpido, incluyendo **PC y PSW**; la rutina también puede guardar los registros que vaya a modificar.
6. Se ejecuta la **rutina de servicio de interrupción (ISR)**.

7. Una vez atendida la interrupción, se envía el **EOI (End of Interrupt)** al PIC para indicar que terminó el servicio y permitir que el PIC actualice su estado y pueda atender nuevas solicitudes.
8. Se restaura el contexto guardado y se realiza el retorno de interrupción, continuando la ejecución del programa desde el punto donde fue interrumpido.

5. Estructura del PIC

Estructura interna del PIC 82C59A

El **PIC 82C59A** es un controlador programable de interrupciones que se coloca entre los dispositivos de E/S y el procesador. Su función es **recibir solicitudes, determinar prioridades, enmascararlas y comunicar al procesador cuál debe atenderse**.

El **IMR (Interrupt Mask Register)** permite enmascarar individualmente las entradas, es decir, impedir que determinadas solicitudes sean atendidas. Luego, el **Priority Resolver** determina cuál de las solicitudes habilitadas tiene mayor prioridad.

La interrupción seleccionada pasa a estar en servicio y se registra en el **ISR (In-Service Register)**, que indica qué interrupción está siendo atendida actualmente. Esto permite gestionar también interrupciones anidadas y las prioridades entre ellas

El PIC se conecta con el procesador mediante las señales **INTR** e **INTA**. Cuando existe una interrupción que debe atenderse, el PIC activa **INTR**. El procesador reconoce la interrupción mediante **INTA**, y el PIC coloca el **vector de interrupción correspondiente en el bus de datos**, permitiendo que el procesador localice la rutina de servicio.

Además, posee lógica de control, lógica de lectura/escritura, un **buffer del bus de datos** y circuitos de **cascada**, que permiten programar el controlador y conectar varios PIC para manejar más fuentes de interrupción. Un PIC maneja ocho entradas directamente y mediante cascada se pueden manejar hasta 64.

6. Diferencia entre una subrutina y un gestor de interrupciones

Una **subrutina** se ejecuta porque el programa realiza explícitamente una llamada, normalmente mediante una instrucción como `CALL`. La llamada guarda la **dirección de retorno**, y el control pasa a la subrutina. La subrutina puede guardar en la pila los registros que necesite preservar y, al finalizar, utiliza una instrucción de retorno como `RET`, que recupera la dirección de retorno y continúa con la instrucción siguiente al `CALL`.

Un **gestor de interrupciones (ISR)** se ejecuta cuando ocurre una interrupción, que puede producirse en cualquier momento durante la ejecución del programa y no es necesariamente provocada por el propio programa. Por eso, al aceptar la interrupción, el procesador debe preservar el estado necesario para poder continuar después. Luego al finalizar se utiliza un retorno de interrupcion como IRET, que permite restaurar el estado del programa interrumpido.

7. ¿Qué método de atención conocemos como interrupción vectorizada?

Una **interrupción vectorizada** es aquella en la que, cuando la CPU reconoce una interrupción, recibe un **vector** que permite identificar o localizar directamente la **rutina de servicio (ISR)** correspondiente al dispositivo que generó la interrupción.

Esto evita que el procesador tenga que recorrer o consultar todos los dispositivos para descubrir cuál generó la interrupción.

Stallings menciona distintas formas de implementar este mecanismo, entre ellas **daisy chain** y **arbitraje de bus**, que permiten que el dispositivo correspondiente proporcione el vector durante el reconocimiento de la interrupción.

Lo importante es que:

interrupción → reconocimiento → vector → ISR correspondiente.

8. ¿Qué es la segmentación de cauce (pipeline)?

La **segmentación de cauce (pipeline)** consiste en dividir la ejecución de las instrucciones en varias etapas, donde cada una realiza una parte del trabajo. De esta manera, se pueden ejecutar varias instrucciones simultáneamente, encontrándose cada una en una etapa diferente.

Por ejemplo, una instrucción puede estar ejecutándose mientras otra se decodifica y una tercera se está captando.

El objetivo es **aumentar la tasa de procesamiento de instrucciones**, aprovechando el paralelismo entre ellas. Una vez que el cauce está lleno, idealmente puede completarse una instrucción por ciclo de reloj.

Sin embargo, pueden aparecer riesgos o retardos por dependencias de datos, saltos o conflictos de recursos, que impiden alcanzar el rendimiento ideal.

9.Rendimiento de la segmentación de cauce

El rendimiento de un **pipeline** se refiere a la mejora que se obtiene al ejecutar varias instrucciones de forma solapada, comparándolo con una ejecución sin segmentación.

En un cauce de **k etapas**, una instrucción tarda aproximadamente k ciclos en recorrer todo el cauce. Pero, una vez que el cauce está lleno, idealmente puede **finalizar una instrucción por ciclo**.

Para ejecutar **n instrucciones**, en condiciones ideales, el tiempo es:

[
T = (k+n-1)\cdot \tau
]

donde (\tau) es el tiempo de ciclo.

Por ejemplo, con 4 etapas y 5 instrucciones, se necesitan (4+5-1=8) ciclos, mientras que sin segmentación serían (4\times5=20) ciclos.

La mejora puede medirse mediante el **speedup**, que compara el tiempo de ejecución sin segmentación con el tiempo de ejecución con pipeline. En condiciones ideales, cuando se ejecutan muchas instrucciones, la aceleración puede acercarse al número de etapas.

En la práctica, el rendimiento es menor debido a **dependencias de datos, riesgos de control y conflictos de recursos**, que pueden generar esperas, burbujas o instrucciones descartadas.

La idea principal es que el pipeline **no necesariamente hace más rápida una instrucción individual**, sino que permite aumentar la cantidad de instrucciones que se completan por unidad de tiempo.

10. Ventajas de la segmentación de cauce

La principal ventaja de la segmentación de cauce (*pipeline*) es que **aumenta el rendimiento del procesador**, permitiendo que varias instrucciones se procesen simultáneamente en distintas etapas.

Entre sus ventajas se encuentran:

- **Mayor tasa de procesamiento:** una vez lleno el cauce, idealmente se completa una instrucción por ciclo.
- **Solapamiento de instrucciones:** mientras una instrucción se ejecuta en una etapa, otras pueden estar en etapas anteriores.
- **Mejor aprovechamiento de los recursos:** las distintas etapas pueden trabajar al mismo tiempo, evitando que los recursos permanezcan inactivos innecesariamente.
- **Posibilidad de aumentar la frecuencia de reloj:** al dividir la ejecución en etapas más pequeñas, puede reducirse el tiempo necesario para cada etapa.

Sin embargo, la mejora real depende de la existencia de riesgos de datos, de control y de recursos, que pueden provocar esperas y disminuir el rendimiento. Además, la segmentación no necesariamente reduce el tiempo de ejecución de una instrucción individual, sino que busca **aumentar la cantidad de instrucciones completadas por unidad de tiempo**.

11. Dependencias de datos en un cauce segmentado

Las dependencias de datos aparecen cuando una instrucción necesita leer o modificar un dato relacionado con otra instrucción. En un cauce segmentado, estas dependencias pueden generar riesgos si las instrucciones se ejecutan simultáneamente y no se respeta el orden necesario para obtener resultados correctos.

Se distinguen tres tipos principales:

- **RAW (Read After Write):** una instrucción necesita leer un dato que una instrucción anterior debe escribir. Es una dependencia verdadera, ya que la segunda instrucción depende del resultado de la primera.
- **WAR (Write After Read):** una instrucción escribe un dato antes de que una instrucción anterior haya terminado de leerlo.
- **WAW (Write After Write):** dos instrucciones escriben en el mismo destino y debe respetarse el orden de escritura para conservar el resultado correcto.

En un cauce simple, las dependencias RAW son las más habituales. Las dependencias WAR y WAW pueden aparecer en arquitecturas que permiten lecturas, escrituras o ejecuciones fuera de orden.

Para resolver estos riesgos se pueden utilizar técnicas como la inserción de esperas (*stalls*), el reenvío de resultados entre etapas (*forwarding*) y el renombrado de registros, según el tipo de riesgo y la arquitectura.

12. **Tres motivos de retardo de un cauce de instrucciones segmentado**

Los retardos de un cauce segmentado se producen cuando una o más instrucciones no pueden avanzar en el momento previsto. Las tres causas principales son:

1. **Riesgos estructurales:** ocurren cuando varias instrucciones necesitan utilizar simultáneamente un recurso que no puede atenderlas a todas. Producen ciclos de espera hasta que el recurso esté disponible. Pueden reducirse utilizando recursos independientes o duplicando recursos cuando sea conveniente.
2. **Riesgos de datos:** aparecen cuando una instrucción necesita un dato que otra instrucción todavía no produjo o no dejó disponible. Producen esperas de la instrucción dependiente hasta obtener el dato necesario. Pueden reducirse mediante técnicas como el reenvío de resultados (*forwarding*).
3. **Riesgos de control:** se producen cuando una instrucción de salto modifica el flujo del programa y aún no se conoce con certeza qué instrucciones deben ejecutarse. Pueden provocar el descarte de instrucciones captadas por el camino incorrecto y la pérdida de ciclos.

Para reducir los retardos por riesgos de control existen varias técnicas:

- **Salto retardado (*delayed branch*):** el compilador coloca una instrucción útil después del salto, en el espacio de retardo. Si no encuentra una adecuada, puede insertar una instrucción NOP, que no realiza una operación útil.
- **Detención del cauce:** el procesador pausa el avance de las instrucciones hasta resolver el salto. Es sencilla, pero puede provocar una pérdida importante de ciclos.
- **Resolución anticipada:** se intenta determinar en una etapa temprana si el salto se toma y cuál es su destino.
- **Predicción de saltos:** se anticipa qué camino seguirá el programa. La predicción estática utiliza una regla fija o una decisión previa; la dinámica utiliza información sobre el comportamiento anterior de los saltos.

En conclusión, los riesgos estructurales, de datos y de control introducen esperas o burbujas en el cauce y disminuyen su rendimiento. Las técnicas de resolución buscan reducir esos retardos y mantener el cauce ocupado tanto como sea posible.

13. Las cuatro variantes de la taxonomía de Flynn

La taxonomía de Flynn clasifica las arquitecturas de computadoras según la cantidad de flujos de instrucciones (la secuencia de instrucciones que ejecuta un procesador) y de datos (los datos sobre los que operan las instrucciones) que procesan. Se distinguen cuatro categorías:

1. **SISD (Single Instruction, Single Data):** posee un único flujo de instrucciones que opera sobre un único flujo de datos. Corresponde al modelo clásico de procesamiento secuencial.
2. **SIMD (Single Instruction, Multiple Data):** una misma instrucción se aplica simultáneamente a varios datos. Se utiliza, por ejemplo, en operaciones vectoriales y procesamiento de imágenes.
3. **MISD (Multiple Instruction, Single Data):** varios flujos de instrucciones operan sobre un mismo flujo de datos. Es una categoría poco frecuente en computadoras de propósito general.
4. **MIMD (Multiple Instruction, Multiple Data):** varios procesadores ejecutan distintos flujos de instrucciones sobre distintos flujos de datos. Permite realizar tareas diferentes en paralelo y se encuentra en sistemas multiprocesador y multinúcleo.

En conclusión, la clasificación se basa en si la arquitectura procesa uno o varios flujos de instrucciones y uno o varios flujos de datos.

14.Comparación entre sistemas SMP y clústeres

Los sistemas **SMP (Symmetric Multiprocessing)** y los **clústeres** son arquitecturas que permiten ejecutar tareas en paralelo, pero se diferencian principalmente en su organización y en la forma de compartir los recursos.

Un sistema SMP posee varios procesadores que comparten una misma memoria principal y los dispositivos de entrada/salida. Los procesadores tienen el mismo nivel de importancia y pueden acceder a los datos compartidos. Entre sus ventajas se encuentran la comunicación relativamente rápida y la facilidad para compartir información. Como desventaja, la memoria y otros recursos compartidos pueden convertirse en cuellos de botella.

Un clúster está formado por varias computadoras independientes, denominadas nodos, conectadas mediante una red para trabajar de manera coordinada. Cada nodo posee sus propios procesadores y memoria principal, y se comunica con los demás a través de la red. Sus principales ventajas son la posibilidad de ampliar la capacidad agregando nodos y de mejorar la disponibilidad mediante mecanismos de tolerancia a fallos. Como desventajas, la comunicación tiene mayor latencia y es necesario coordinar el trabajo entre las computadoras.

En conclusión, **SMP utiliza varios procesadores que comparten una misma memoria, mientras que un clúster combina varias computadoras con memorias independientes**

conectadas mediante una red. SMP facilita la comunicación entre procesadores; un clúster ofrece mayor flexibilidad para ampliar la capacidad y distribuir el trabajo entre nodos.

15. **MIMD y organización de la memoria**

MIMD (*Multiple Instruction, Multiple Data*) es una categoría de la taxonomía de Flynn en la que varios procesadores ejecutan diferentes flujos de instrucciones sobre diferentes flujos de datos de manera concurrente. Esto permite realizar tareas distintas en paralelo.

Las arquitecturas MIMD pueden organizar la memoria de dos formas principales:

- **Memoria compartida:** los procesadores acceden a una misma memoria principal y pueden intercambiar información mediante datos compartidos. Un ejemplo es un sistema SMP.
- **Memoria distribuida:** cada procesador o nodo dispone de su propia memoria local y se comunica con los demás mediante el intercambio de mensajes a través de una red. Un ejemplo es un clúster.

En conclusión, MIMD permite el procesamiento paralelo mediante múltiples flujos de instrucciones y datos. Puede implementarse con memoria compartida o distribuida, según la organización del sistema.

16. **Características de los procesadores superescalares**

Un procesador superescalar es aquel que puede emitir e iniciar la ejecución de varias instrucciones por ciclo de reloj, aprovechando el paralelismo a nivel de instrucciones (ILP).

Sus principales características son:

- **Ejecución paralela:** permite procesar simultáneamente varias instrucciones independientes.
- **Múltiples unidades funcionales:** dispone de recursos de ejecución que pueden trabajar en paralelo, como unidades aritméticas, de coma flotante y de carga/almacenamiento.
- **Detección de dependencias:** identifica las relaciones entre instrucciones para evitar resultados incorrectos.
- **Planificación y emisión de instrucciones:** selecciona las instrucciones que están listas para ejecutarse. Según el diseño, la emisión puede realizarse en orden o fuera de orden.
- **Técnicas de optimización:** puede utilizar predicción de saltos, ejecución especulativa y renombrado de registros para reducir los retardos y aprovechar mejor los recursos.

El rendimiento de un procesador superescalar depende de la cantidad de instrucciones independientes disponibles, de sus dependencias, de los recursos de ejecución y de los riesgos estructurales y de control. En conclusión, la arquitectura superescalar busca aumentar la cantidad de instrucciones procesadas por unidad de tiempo ejecutando varias instrucciones en paralelo, sin modificar el resultado que debe producir el programa.

17. **Políticas de emisión de instrucciones en procesadores superescalares**

Las políticas de emisión determinan en qué orden las instrucciones pasan a ejecutarse y en qué orden finalizan. Se distinguen tres tipos principales:

1. **Emisión en orden con finalización en orden:** las instrucciones se emiten y finalizan respetando el orden del programa. Es una política sencilla, pero puede provocar esperas cuando una instrucción demora la ejecución de las siguientes.
2. **Emisión en orden con finalización fuera de orden:** las instrucciones se emiten en el orden del programa, pero pueden finalizar en otro orden. Esto permite aprovechar mejor las unidades funcionales, aunque deben controlarse las dependencias para garantizar resultados correctos.
3. **Emisión fuera de orden con finalización fuera de orden:** las instrucciones pueden emitirse y finalizar en un orden diferente al del programa, siempre que se respeten las dependencias y los conflictos. Para ello, puede utilizarse una ventana de instrucciones que permite seleccionar las instrucciones listas para ejecutarse, incluso si otras anteriores deben esperar. Esta política aumenta las posibilidades de paralelismo, pero requiere hardware y mecanismos de control más complejos.

En conclusión, las políticas más flexibles buscan reducir las esperas y aprovechar mejor las unidades funcionales del procesador, manteniendo la ejecución correcta del programa.

18. **Elección y justificación de una política de emisión**

Elegiría la política de **emisión fuera de orden con finalización fuera de orden**, porque permite aprovechar mejor el paralelismo a nivel de instrucciones y mejorar el rendimiento del procesador.

Esta política permite emitir instrucciones posteriores antes que otras anteriores cuando están listas para ejecutarse y no existen dependencias ni conflictos que lo impidan. De esta manera, si una instrucción debe esperar por un dato o por un recurso, otras instrucciones independientes pueden avanzar en paralelo, reduciendo los ciclos de espera.

Para implementar esta política se puede utilizar una ventana de instrucciones, que permite mantener varias instrucciones disponibles y seleccionar aquellas que pueden ejecutarse. Sin embargo, requiere una lógica de control más compleja para administrar las dependencias, los conflictos y el orden de los resultados.

En conclusión, elegiría esta política porque ofrece mayores posibilidades de aprovechar las unidades funcionales y aumentar el rendimiento, aunque a costa de una mayor complejidad del hardware.

19.**¿De qué depende el paralelismo en un procesador superescalar?**

El paralelismo en un procesador superescalar depende principalmente de la cantidad de **instrucciones independientes disponibles para ejecutarse simultáneamente**.

Los factores que influyen son:

- **Dependencias de datos:** limitan el paralelismo cuando una instrucción necesita el resultado de otra anterior.
- **Cantidad de unidades funcionales:** determina cuántas instrucciones pueden ejecutarse simultáneamente, según el tipo de operación.
- **Riesgos de control:** los saltos pueden dificultar la identificación de las instrucciones que deben ejecutarse. La predicción de saltos permite reducir estos inconvenientes.
- **Política de emisión:** la emisión fuera de orden permite seleccionar instrucciones disponibles aunque otras anteriores deban esperar.
- **Renombrado de registros:** elimina ciertas dependencias artificiales y permite aumentar las posibilidades de ejecución paralela.

Además, el paralelismo depende de las características del programa y de las técnicas de optimización utilizadas por el compilador y el hardware.

En conclusión, aunque un procesador tenga múltiples unidades funcionales, su rendimiento depende de que existan suficientes instrucciones independientes y de que la arquitectura pueda aprovecharlas sin producir resultados incorrectos.

20. Objetivo del renombrado de registros

El objetivo del **renombrado de registros** es eliminar las dependencias artificiales entre instrucciones que utilizan el mismo registro, aumentando las posibilidades de ejecución paralela en un procesador superescalar.

Para ello, el procesador asigna registros físicos diferentes a los resultados que, desde el punto de vista del programa, utilizan un mismo registro lógico. De esta manera, se evitan conflictos innecesarios entre instrucciones independientes.

Esta técnica permite eliminar dos tipos de dependencias:

- **WAR (Write After Read):** una escritura posterior puede interferir con la lectura de una instrucción anterior.
- **WAW (Write After Write):** dos instrucciones escriben en el mismo registro y podrían producir resultados incorrectos si no se respeta el orden adecuado.

El renombrado no elimina las dependencias **RAW (Read After Write)**, porque representan dependencias verdaderas: una instrucción necesita el resultado producido por otra.

En conclusión, el renombrado de registros reduce las dependencias artificiales, facilita la emisión y ejecución fuera de orden y permite aprovechar mejor el paralelismo a nivel de instrucciones.

21. ¿Qué es el paralelismo y de qué depende?

El **paralelismo** consiste en ejecutar simultáneamente varias operaciones o tareas con el objetivo de reducir el tiempo de ejecución o aumentar el rendimiento del sistema.

El paralelismo depende principalmente de:

- **Las características del programa:** deben existir operaciones o tareas independientes que puedan ejecutarse al mismo tiempo.
- **Los recursos del sistema:** la cantidad de procesadores, núcleos y unidades funcionales disponibles.
- **Las dependencias entre operaciones:** cuando una operación necesita el resultado de otra, su ejecución debe respetar esa dependencia.
- **Los recursos de memoria y comunicación:** los conflictos por recursos compartidos, los accesos a memoria y la sincronización pueden limitar el rendimiento.

Existen diferentes niveles de paralelismo, como el paralelismo a nivel de instrucciones, de datos y de tareas. Cada uno aprovecha distintas posibilidades de ejecución simultánea.

En conclusión, el rendimiento depende de dos aspectos: el paralelismo que permite el programa y la capacidad del hardware para aprovecharlo. Aunque un procesador tenga varios cauces de ejecución, no podrá utilizarlos plenamente si el programa presenta muchas dependencias.

22. ¿Por qué funciona un sistema de memoria basado en jerarquía?

Un sistema de memoria basado en jerarquía funciona gracias al **principio de localidad de las referencias** y al compromiso entre capacidad, tiempo de acceso y costo por bit de las distintas tecnologías de memoria.

Al descender en la jerarquía, disminuye el costo por bit, aumenta la capacidad y aumenta el tiempo de acceso. Sin embargo, también disminuye la frecuencia con la que el procesador necesita acceder a los niveles inferiores.

El principio de localidad establece que las referencias a memoria tienden a concentrarse en determinadas posiciones durante intervalos de tiempo. Se distinguen dos tipos:

- **Localidad temporal:** si una instrucción o dato se utilizó recientemente, existe una alta probabilidad de que vuelva a utilizarse próximamente.
- **Localidad espacial:** si se accede a una posición de memoria, existe una alta probabilidad de acceder próximamente a posiciones cercanas.

Gracias a estas propiedades, los datos e instrucciones utilizados con mayor frecuencia pueden permanecer en los niveles superiores, más rápidos y pequeños. Cuando se produce un acierto (*hit*), la información se obtiene desde ese nivel; cuando ocurre un fallo (*miss*), debe recuperarse desde un nivel inferior.

De esta manera, la mayoría de los accesos pueden resolverse en memorias rápidas, reduciendo el tiempo medio de acceso sin necesitar que toda la memoria tenga un costo y una velocidad elevados.

En conclusión, la jerarquía de memoria permite combinar gran capacidad, bajo costo y buen rendimiento porque aprovecha la localidad de las referencias para reducir la frecuencia de acceso a los niveles más lentos.

23.**Políticas de escritura y coherencia de datos**

Las políticas de escritura de la memoria caché determinan cómo y cuándo se actualizan los datos en la caché y en la memoria principal. Las dos políticas principales son:

1. Escritura inmediata (*write-through*): cada escritura actualiza tanto la caché como la memoria principal. Su ventaja es que la memoria principal permanece actualizada con respecto a esa caché. Su desventaja es que genera mayor tráfico de memoria y puede disminuir el rendimiento.

2. Escritura diferida (*write-back*): las modificaciones se realizan inicialmente solo en la caché. Se utiliza un bit de modificación (*dirty bit*) para identificar los bloques modificados. Cuando un bloque debe reemplazarse, se escribe en la memoria principal únicamente si fue modificado. Esta política reduce las escrituras a memoria y puede mejorar el rendimiento, pero la memoria principal puede contener temporalmente datos desactualizados.

En sistemas con varios procesadores y cachés, estas políticas plantean un problema de **coherencia de caché**, porque distintas cachés pueden conservar copias del mismo dato y alguna de ellas quedar desactualizada. Incluso con *write-through*, otras cachés pueden conservar valores antiguos.

Stallings presenta distintos mecanismos para mantener la coherencia:

- **Vigilancia del bus (*bus watching*):** los controladores observan las escrituras de otros agentes y pueden invalidar las copias afectadas.
- **Transparencia por hardware:** se propagan las actualizaciones para mantener actualizadas las copias correspondientes en las distintas cachés.
- **Memoria no cacheable:** las regiones de memoria compartida se excluyen de las cachés, evitando el problema de las copias desactualizadas a costa de perder rendimiento en esos accesos.

En conclusión, *write-through* facilita la coherencia con la memoria principal, pero genera más tráfico; *write-back* reduce las escrituras y mejora potencialmente el rendimiento, pero requiere mecanismos adicionales para garantizar la coherencia entre las distintas copias de los datos.

24. **Ventajas y desventajas de varios niveles de memoria caché**

Los sistemas con varios niveles de memoria caché combinan memorias pequeñas y rápidas con otras de mayor capacidad y menor velocidad. Por ejemplo, L1 es generalmente la más rápida, mientras que L2 y L3 ofrecen mayor capacidad.

Ventajas:

- Reducen el tiempo medio de acceso a memoria, porque los datos pueden encontrarse en alguno de los niveles de caché antes de acceder a la memoria principal.
- Disminuyen los accesos a memoria principal y el tráfico por el bus.

- Permiten combinar alta velocidad de acceso con una capacidad total de caché mayor.

Desventajas:

- Aumentan el costo, el consumo de energía y la complejidad del hardware.
- Requieren administrar cada nivel, incluyendo las políticas de reemplazo y escritura.
- Cuando ocurre un fallo en un nivel, es necesario consultar los niveles inferiores, agregando latencia respecto de un acierto en L1.

En conclusión, varios niveles de caché pueden mejorar el rendimiento al reducir los accesos a memoria principal, aunque incorporan mayor complejidad y costo.

25. Funciones de correspondencia entre memoria principal y caché

Como la memoria principal contiene más bloques que los que puede almacenar la caché, es necesario establecer una función de correspondencia que determine dónde puede ubicarse cada bloque. Existen tres métodos principales:

1. Correspondencia directa: cada bloque de memoria principal puede ubicarse en una única línea de caché. Es sencilla y económica de implementar, pero puede producir conflictos si varios bloques corresponden a la misma línea.

2. Correspondencia asociativa: cualquier bloque de memoria principal puede ubicarse en cualquier línea de caché. Ofrece mayor flexibilidad y reduce las restricciones de ubicación, pero necesita comparar las etiquetas de las líneas para localizar un bloque, lo que aumenta la complejidad del hardware.

3. Correspondencia asociativa por conjuntos: la caché se divide en conjuntos con varias líneas. Cada bloque de memoria principal corresponde a un conjunto específico y puede ubicarse en cualquiera de sus líneas. Este método combina características de la correspondencia directa y de la asociativa.

En conclusión, la correspondencia directa prioriza la simplicidad, la asociativa ofrece máxima flexibilidad y la asociativa por conjuntos constituye una solución intermedia.

26.Elementos de diseño de una memoria caché

Según Stallings, los principales elementos de diseño de una memoria caché son:

1. **Tamaño de la caché:** determina la cantidad de información que puede almacenarse. Una mayor capacidad puede reducir los fallos, aunque incrementa el costo y puede aumentar el tiempo de acceso.
2. **Función de correspondencia:** establece dónde puede ubicarse cada bloque de memoria principal. Puede ser directa, asociativa o asociativa por conjuntos.
3. **Algoritmo de reemplazo:** determina qué bloque se elimina cuando es necesario cargar otro y no hay una línea libre en la ubicación correspondiente. Los principales algoritmos son:

    - **LRU (Least Recently Used):** reemplaza el bloque que lleva más tiempo sin ser utilizado. Se basa en que los bloques utilizados recientemente tienen más probabilidades de volver a utilizarse.
    - **FIFO (First In, First Out):** reemplaza el bloque que lleva más tiempo en la caché, independientemente de cuándo fue utilizado por última vez.
    - **LFU (Least Frequently Used):** reemplaza el bloque que tuvo menos referencias o accesos. Puede implementarse utilizando un contador para cada línea.
    - **Aleatorio (Random):** selecciona al azar uno de los bloques candidatos al reemplazo.
4. En la correspondencia directa no se necesita elegir un bloque, porque cada bloque de memoria principal solo puede ocupar una línea determinada. En las correspondencias asociativa y asociativa por conjuntos sí se necesita una política de reemplazo cuando no hay espacio libre.
5. **Política de escritura:** define cuándo se actualizan la caché y la memoria principal. Entre las principales políticas se encuentran *write-through*, que actualiza ambas en cada escritura, y *write-back*, que actualiza inicialmente la caché y posterga la escritura en memoria principal hasta que sea necesario.
6. **Tamaño de línea:** establece cuántos datos se transfieren entre la memoria principal y la caché en cada carga de bloque. Debe elegirse considerando la localidad espacial y el costo de transferir información.
7. **Número y organización de cachés:** se puede utilizar un único nivel o varios niveles, así como cachés unificadas o separadas para instrucciones y datos.
8. **Direcciones de caché:** la caché puede utilizar direcciones lógicas o físicas, lo que influye en su organización y en su interacción con la unidad de gestión de memoria.

En conclusión, el diseño de la caché busca equilibrar el tiempo de acceso, la tasa de aciertos, la capacidad, el costo y la complejidad del sistema.

27. **Mejor tiempo de acceso mediante la memoria caché**

Para obtener un buen tiempo medio de acceso mediante la memoria caché, se deben elegir adecuadamente sus elementos de diseño, buscando reducir tanto el tiempo de acceso cuando hay un acierto como la frecuencia y el costo de los fallos.

- **Tamaño de la caché:** debe elegirse un tamaño equilibrado. Una mayor capacidad puede reducir los fallos, pero también puede aumentar el tiempo de acceso y el costo. Por eso, conviene utilizar una caché pequeña y rápida en el nivel más cercano al procesador y niveles inferiores de mayor capacidad.
- **Función de correspondencia:** la correspondencia directa permite localizar los bloques de manera sencilla y rápida, pero puede producir más fallos por conflictos. La correspondencia asociativa reduce las restricciones de ubicación, aunque necesita hardware más complejo para comparar las etiquetas. La correspondencia asociativa por conjuntos representa un compromiso entre rapidez, flexibilidad y cantidad de fallos.
- **Algoritmo de reemplazo:** debe procurar conservar los bloques que probablemente vuelvan a utilizarse. LRU reemplaza el bloque menos recientemente utilizado y Stallings lo presenta como una alternativa eficaz para conseguir una buena tasa de

aciertos. FIFO, LFU y el reemplazo aleatorio son otras posibilidades. La elección influye principalmente en la tasa de fallos y en la complejidad del hardware.
- **Política de escritura:** *write-through* actualiza la caché y la memoria principal en cada escritura, lo que facilita mantener actualizada la memoria principal, pero genera más tráfico. *Write-back* posterga la actualización de la memoria principal hasta que se reemplaza un bloque modificado, reduciendo las escrituras y pudiendo mejorar el rendimiento, aunque requiere mecanismos adicionales de control y coherencia.
- **Tamaño de línea o bloque:** debe aprovechar la localidad espacial sin transferir datos innecesarios. Los bloques demasiado pequeños pueden desaprovechar accesos a posiciones cercanas; los demasiado grandes aumentan el tiempo de transferencia y pueden reducir la cantidad de bloques diferentes que caben en la caché.
- **Número y organización de las cachés:** utilizar varios niveles permite combinar una caché L1 pequeña y rápida con niveles L2 y L3 de mayor capacidad. Separar la caché de instrucciones de la de datos también puede permitir accesos simultáneos y reducir conflictos entre ambos tipos de acceso.
- **Direcciones de caché:** una caché que comienza la búsqueda utilizando direcciones lógicas puede iniciar el acceso antes de que termine la traducción a dirección física, reduciendo potencialmente la latencia inicial. Sin embargo, presenta dificultades adicionales de organización y coherencia respecto de una caché física.

En conclusión, para mejorar el tiempo medio de acceso se debe buscar una tasa de aciertos elevada, un tiempo de acceso reducido y una penalización por fallo lo menor posible. Una organización habitual consiste en utilizar una L1 pequeña y rápida, asociatividad moderada, un tamaño de bloque equilibrado y niveles inferiores de mayor capacidad. La configuración óptima depende de las características del programa y de los costos de cada alternativa.

28. Estructura de un módulo de entrada/salida

Un módulo de entrada/salida (E/S) es el componente que permite la comunicación entre el procesador y la memoria principal, por un lado, y los dispositivos periféricos, por el otro. Se conecta al sistema mediante el bus de datos, el bus de direcciones y las líneas de control.

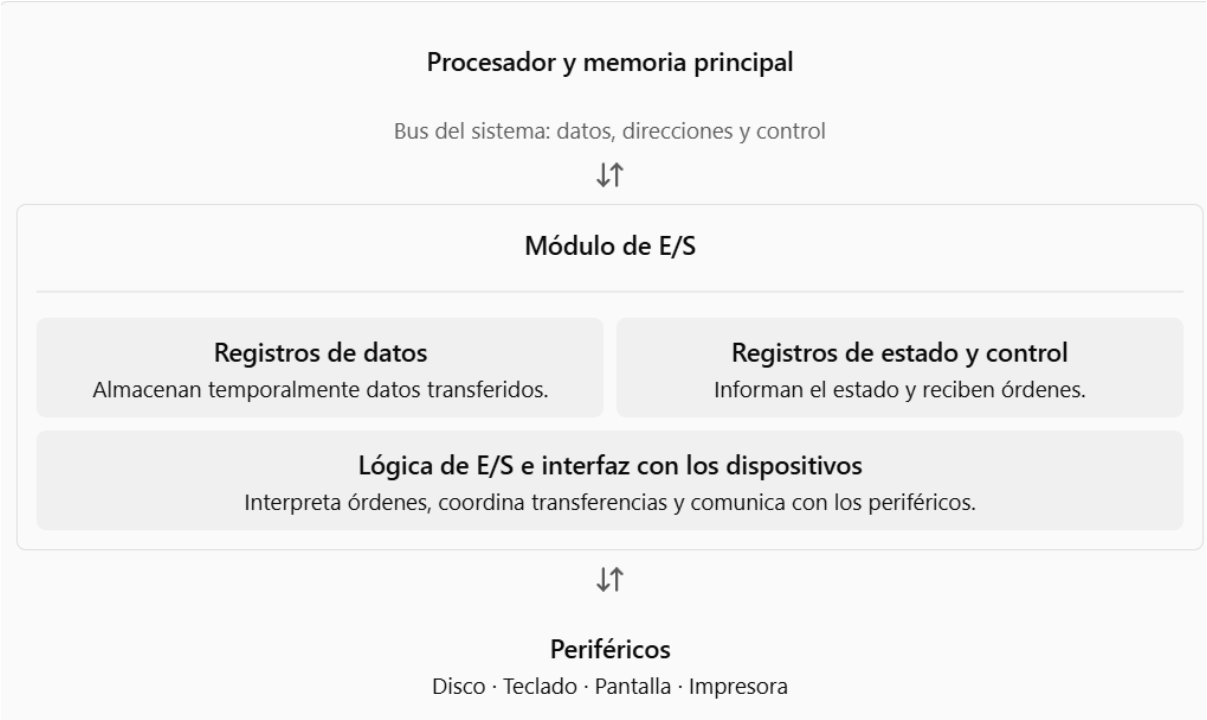
Su estructura general comprende:

- **Interfaz con el bus del sistema:** permite recibir direcciones, intercambiar datos y recibir órdenes del procesador.
- **Registros de datos:** almacenan temporalmente los datos que se transfieren entre el sistema y los periféricos.
- **Registros de estado y control:** informan si el dispositivo está listo, ocupado o presenta errores, y permiten configurar y controlar las operaciones.
- **Lógica de E/S:** interpreta las órdenes, reconoce las direcciones correspondientes y coordina las transferencias.
- **Interfaz con los dispositivos externos:** permite intercambiar datos, órdenes y señales de estado con uno o varios periféricos.

Las principales funciones de un módulo de E/S son controlar y sincronizar las operaciones, comunicarse con el procesador y los dispositivos, realizar el almacenamiento temporal de datos (*buffering*) y detectar e informar errores.

Stallings también distingue cuatro tipos de órdenes que puede recibir el módulo: control, prueba del estado (*test*), lectura (*read*) y escritura (*write*).

En conclusión, el módulo de E/S adapta y coordina la comunicación entre el sistema y los periféricos, evitando que el procesador deba gestionar directamente todos los detalles de cada dispositivo.



29.Coherencia de un sistema jerárquico y su relación con DMA

La **coherencia de memoria** consiste en garantizar que las copias de un mismo dato presentes en los distintos niveles de la jerarquía se mantengan consistentes. Si un dato se modifica, las demás copias deben actualizarse o invalidarse según el mecanismo utilizado, de modo que no se utilicen valores obsoletos.

Se puede distinguir entre coherencia vertical, que relaciona distintos niveles de memoria, y coherencia horizontal, que aparece cuando varias cachés del mismo nivel poseen copias de un mismo bloque.

El uso de **DMA (acceso directo a memoria)** puede afectar la coherencia, porque permite que un dispositivo de E/S lea o escriba directamente en la memoria principal sin pasar por el procesador ni necesariamente por su caché.

Esto puede generar dos problemas:

- Si el procesador modificó un dato en una caché *write-back* y todavía no lo escribió en memoria principal, el DMA puede leer el valor anterior.
- Si el DMA modifica la memoria principal, una caché puede conservar una copia antigua que el procesador lea posteriormente.

Para evitar estos problemas se utilizan mecanismos de coherencia implementados por hardware o tareas de mantenimiento de caché, como escribir los datos modificados en memoria principal (*clean*) e invalidar las copias que puedan quedar desactualizadas (*invalidate*). El mecanismo concreto depende de la arquitectura.

En conclusión, el DMA puede afectar la coherencia si las transferencias no se coordinan con las cachés. El sistema debe garantizar que tanto el procesador como los dispositivos utilicen la versión válida de los datos.

30.Acceso directo a memoria (DMA) y sus ventajas

El **DMA (Direct Memory Access)** es una técnica de entrada/salida que permite transferir bloques de datos directamente entre un dispositivo periférico y la memoria principal, sin que el procesador tenga que intervenir en cada transferencia individual.

Para iniciar la operación, el procesador configura el controlador DMA con la dirección inicial de memoria, la cantidad de datos que deben transferirse y el sentido de la operación. Luego, el controlador solicita el acceso al bus y realiza la transferencia directamente entre el dispositivo y la memoria. Cuando finaliza, normalmente genera una interrupción para informar al procesador.

La principal ventaja del DMA es que **reduce la intervención del procesador y permite transferir grandes volúmenes de datos de manera más eficiente**. A diferencia de la E/S programada y de la E/S por interrupciones convencionales, no requiere que el procesador ejecute instrucciones para mover cada palabra entre el módulo de E/S y la memoria.

De esta manera, el procesador puede dedicarse a otras tareas durante la transferencia. Sin embargo, el DMA puede competir con el procesador por el uso del bus y de la memoria principal, y requiere un tiempo inicial de configuración.

En conclusión, el DMA mejora la eficiencia de las operaciones de E/S, especialmente en transferencias de bloques grandes, al delegar el movimiento de datos en un controlador especializado y reducir la carga de trabajo del procesador.

31. Mecanismo de interrupciones

Una interrupción es un mecanismo que permite suspender temporalmente la ejecución normal del procesador para atender un evento que requiere servicio, como una solicitud de un dispositivo de E/S.

El mecanismo general comprende los siguientes pasos:

1. **Solicitud:** un dispositivo o controlador genera una solicitud de interrupción.
2. **Reconocimiento:** el procesador termina la instrucción en curso y comprueba si existe una interrupción pendiente que pueda atender, considerando las habilitaciones y prioridades correspondientes.
3. **Salvado del contexto:** el procesador suspende el programa actual y guarda la información necesaria para reanudarlo, como la dirección de la próxima instrucción y el estado del procesador. Los registros adicionales deben preservarse según las necesidades de la rutina y la arquitectura.
4. **Identificación y transferencia de control:** el procesador determina qué evento debe atender y transfiere la ejecución a la rutina de servicio de interrupción (ISR).
5. **Atención:** la ISR realiza las operaciones necesarias, como consultar el estado del dispositivo, transferir datos o limpiar la causa de la solicitud. Si se utiliza un controlador de interrupciones programable (PIC), puede ser necesario enviarle el comando de fin de interrupción (EOI), según su configuración.
6. **Restauración y retorno:** se restaura el contexto guardado y se retorna al programa interrumpido, que continúa su ejecución.

Las interrupciones mejoran el aprovechamiento del procesador porque permiten ejecutar otras instrucciones mientras un dispositivo realiza una operación y solicitar atención únicamente cuando resulta necesario.

En conclusión, el mecanismo conserva el estado del programa interrumpido, transfiere el control a la rutina correspondiente y permite reanudar la ejecución una vez atendido el evento.

32. Diferencias entre una subrutina y un gestor de interrupciones

Una **subrutina** se ejecuta cuando el programa realiza una llamada explícita, mediante una instrucción como `CALL`. La dirección de retorno se guarda mediante el mecanismo definido por la arquitectura, por ejemplo, en la pila o en un registro de enlace. Al finalizar, la instrucción de retorno (`RET`) devuelve el control a la instrucción siguiente a la llamada. La subrutina preserva los registros que necesita según la convención de llamada y sus propias necesidades.

En cambio, un **gestor o rutina de servicio de interrupción (ISR)** se ejecuta como consecuencia de una solicitud de interrupción, generalmente originada por un evento externo al flujo normal del programa. El procesador guarda inicialmente el estado necesario para poder reanudar la ejecución, incluyendo la dirección de retorno y la información de estado correspondiente. La rutina también debe preservar los registros que modifica y que deban conservarse.

Al finalizar, el gestor utiliza un mecanismo de retorno de interrupción, que restaura el estado guardado y permite continuar el programa interrumpido desde el punto correcto.

En conclusión, la subrutina se invoca explícitamente mediante una llamada del programa y retorna a la instrucción siguiente a esa llamada. El gestor de interrupciones se ejecuta ante un evento y, al finalizar, debe restaurar el contexto necesario para reanudar correctamente el programa interrumpido.

33. Interrupciones vectorizadas

Las **interrupciones vectorizadas** son un método de atención en el que, al reconocerse una interrupción, se obtiene un vector que permite identificar o localizar directamente la rutina de servicio correspondiente al dispositivo que la generó.

El vector puede contener la dirección de la rutina o un identificador que permita obtenerla. De esta manera, se evita que el procesador tenga que ejecutar primero una rutina general para consultar uno por uno los dispositivos y determinar cuál solicitó atención.

Stallings describe dos técnicas para implementar interrupciones vectorizadas:

- **Cadena de reconocimiento (*daisy chain*):** los módulos de E/S comparten una solicitud de interrupción. Cuando el procesador envía la señal de reconocimiento, esta se propaga por la cadena hasta alcanzar al dispositivo solicitante, que proporciona el vector por el bus de datos. El orden de los módulos también puede establecer prioridades.
- **Arbitraje del bus:** el módulo solicitante obtiene primero el control del bus. Cuando el procesador reconoce la interrupción, el módulo correspondiente proporciona el vector que permite localizar su rutina de servicio.

En conclusión, las interrupciones vectorizadas agilizan la identificación del dispositivo y la transferencia de control a su rutina específica, reduciendo el trabajo necesario para reconocer el origen de la interrupción.

34. Formas de realizar operaciones de entrada/salida

Las operaciones de E/S pueden realizarse mediante tres técnicas principales:

1. **E/S programada:** el procesador envía una orden al módulo de E/S y consulta su estado hasta que el dispositivo está listo. Participa directamente en la transferencia de cada dato, por lo que puede desperdiciar tiempo en la espera.
2. **E/S por interrupciones:** el procesador inicia la operación y continúa ejecutando otras instrucciones. Cuando el módulo de E/S necesita atención, solicita una interrupción. El procesador ejecuta la rutina de servicio y participa en la transferencia de los datos.
3. **Acceso directo a memoria (DMA):** el procesador configura la operación, pero el controlador DMA transfiere directamente los datos entre el dispositivo y la memoria principal. Al finalizar, normalmente notifica al procesador mediante una interrupción.

En conclusión, estas técnicas se diferencian por la forma en que se detecta la disponibilidad del dispositivo y por el grado de participación del procesador en la transferencia. La E/S programada utiliza consulta continua, la E/S por interrupciones avisa cuando se requiere atención y el DMA permite transferir bloques sin que el procesador mueva cada dato individualmente.

35. Interrupciones de software: cómo y cuándo se utilizan

Las **interrupciones de software** se generan mediante la ejecución de una instrucción específica que provoca una transferencia de control hacia una rutina de atención. A diferencia de las interrupciones de hardware, no se originan directamente en una señal externa de un dispositivo.

Se utilizan cuando un programa necesita solicitar un servicio del sistema operativo o transferir deliberadamente el control a una rutina determinada. También pueden emplearse para comunicar o atender determinadas condiciones de error, según la arquitectura.

El mecanismo consiste en ejecutar la instrucción de interrupción, guardar el estado necesario para retomar la ejecución, transferir el control a la rutina correspondiente y, una vez atendido el pedido, restaurar el estado y devolver el control al programa.

36.Coherencia de la jerarquía de memoria y DMA

La coherencia consiste en mantener consistentes las copias de un mismo dato almacenadas en la caché y en la memoria principal, evitando que el procesador o un dispositivo utilicen valores desactualizados.

Sí, el DMA puede afectar la coherencia, ya que permite transferir datos directamente entre un dispositivo de E/S y la memoria principal sin que el procesador intervenga en cada transferencia ni se consulte necesariamente su caché.

Pueden presentarse dos situaciones:

- Si el procesador modifica un dato en una caché con política *write-back*, la memoria principal puede conservar el valor anterior. Si el DMA lee directamente esa memoria, obtendrá un dato desactualizado.
- Si el DMA escribe un dato nuevo en la memoria principal, una caché puede conservar una copia antigua. Si el procesador lee esa copia sin actualizarla o invalidarla, obtendrá un valor incorrecto.

Para evitar estos problemas, se pueden utilizar mecanismos de coherencia por hardware o tareas de mantenimiento de caché, como escribir en memoria principal los bloques modificados (*clean*) e invalidar las copias que hayan quedado desactualizadas (*invalidate*).

En conclusión, el DMA puede afectar la coherencia si sus transferencias no se coordinan con las cachés. El sistema debe garantizar que el procesador y los dispositivos accedan a la versión válida de los datos.

37.Elementos que definen un bus

Un **bus** es un medio de comunicación que conecta dos o más componentes de una computadora y permite transferir datos, direcciones y señales de control.

Los principales elementos que permiten caracterizarlo son:

1. **Tipo de bus:** puede ser dedicado, cuando las líneas tienen una función específica, o multiplexado, cuando las mismas líneas se utilizan para distintas funciones en diferentes momentos.
2. **Método de arbitraje:** determina qué dispositivo obtiene el control del bus cuando varios lo solicitan. Puede ser centralizado, si existe un controlador que asigna el acceso, o distribuido, si los dispositivos participan en la decisión.
3. **Temporización:** puede ser síncrona, cuando las operaciones se coordinan mediante una señal de reloj, o asíncrona, cuando se coordinan mediante señales de control sin depender de un reloj común.
4. **Ancho del bus:** el ancho del bus de datos determina cuántos bits se transmiten simultáneamente, mientras que el ancho del bus de direcciones determina cuántas posiciones de memoria pueden identificarse.
5. **Tipo de transferencia:** define las operaciones que admite el bus, como lectura, escritura, lectura-modificación-escritura, lectura después de escritura y transferencia por bloques.

En conclusión, estos elementos permiten caracterizar un bus según cómo utiliza sus líneas, cómo asigna el acceso, cómo coordina las operaciones, cuántos bits maneja y qué tipos de transferencias admite.